Pumice: Eroding GPU Fingerprints Through Randomization of WebGL Buffers

Rohit Chatterjee, Noelle Crawford
University of Illinois at Urbana-Champaign
{rohitc2,noellec3}@illinois.edu

Abstract

User privacy on the internet is of paramount importance, especially with the pervasive collection and storage of personal data. This paper focuses on mitigating the susceptibility of browsers to GPU fingerprinting, a type of browser fingerprinting which exploits differences in GPU execution times. By modifying WebGL to reduce the visibility sites have into the GPU hardware, we reduce their ability to craft a unique and useful fingerprint. Results show a clear difference in the deviation of different GPU fingerprints before and after implementing our design, validating our approach in mitigating browser susceptibility to GPU fingerprinting. This work contributes to the ongoing efforts to prioritize user privacy and improve the online experience.

1 Introduction

The significance of user privacy is paramount, particularly in light of the internet's pervasive integration into our daily lives. The fundamental right to privacy ought to be accorded to individuals, given the salience of this issue. In the current digital age, the collection and storage of personal data has become an essential component of online services and platforms. This has resulted in an unprecedented proliferation of user data, which can include sensitive information such as names, email addresses, phone numbers, and financial data. Companies and organizations frequently utilize this data for a variety of purposes, including targeted advertising, market research, and decision-making processes that can significantly impact individuals' lives

The protection of personal data on the internet is of great importance to users for several compelling reasons. Firstly, unauthorized access to personal data can result in identity theft, financial fraud, and other types of cybercrime that can have far-reaching and severe consequences for individuals. Additionally, the use of personal data by companies and organizations can lead to intrusive and manipulative advertising practices that undermine individuals' privacy and autonomy.

Unfortunately, the collection of user data is a massive industry, generating over $150 billion in revenue every year[4], a number that continues to grow. Recent reports estimate that the data generated from a single adult is worth about $35/month[3], giving companies the opportunity to brush off user privacy for profit. Data recorded based on users online activity can contain information about who they are, what they like, and where they live (amongst other things), which is incredibly valuable.

Typically, we assume this data is being recorded through browser cookies. Through the use of cookies, websites are able to uniquely identify users and associate behavior across multiple sessions with a single identity. The introduction of GDPR (the General Data Protection Regulation) undermines the ability of websites to track users in this way by requiring them to first acquire user consent. This is a large step towards prioritizing user privacy, but it does not go far enough.

Unfortunately, cookies are only the *most transparent* form of web user tracking. Sites are still able to employ techniques such as user ip tracking and browser fingerprinting in order to uniquely identify visitors to their sites. IP tracking This method of identification can be circumvented, through the use of a VPN, but such efforts require the use of external servers, which often imposes a financial burden on the user. It is also harder to detect than cookies, which are declared 'transparently' in the browser.

Browser fingerprinting, aka device fingerprinting is the process of identifying users based on browser and hardware configurations. This strategy is the most difficult to circumvent, since the browser must have access to this information in order to access certain features. This can be done using the APIs provided for the browser's use such as the canvas, audio, and WebGL. The site *nothingprivate.ml*[1] demonstrates this technique by tracking the names of users across browser sessions, even with incognito mode and a VPN.

The practice of browser fingerprinting raises considerable concerns due to the imperceptible nature of its implementation. As users have no means of ascertaining whether a website employs this technique, their online activity can be monitored and tracked without their knowledge or consent. Unfortunately, there is no good way to disable browser fingerprinting techniques, and attempting to circumvent them would requires users to forgo certain website functionalities, thereby negatively impacting their online experience.

In this project we aim to mitigate browser's susceptibility to GPU fingerprinting, which is the process of differentiating between browsers due to differences between GPUs. To do this, we modify WebGL to prevent sites from being able to explicitly assign tasks to specific execution units. This prevents attackers from being able to accurately measure the minute differences in the execution units from manufacturing, and reduces the number of data points useable to create

the fingerprint to a single value (average overall execution time) with much higher standard deviation.

2 Background

2.1 FP-Stalker

FP-stalker is a browser fingerprinting tool that tracks browser fingerprints and their evolution over periods of time[5]. The paper finds that by linking multiple characteristics into a single fingerprint, they can achieve much higher accuracy as compared to previous fingerprinting tools, especially when attempting to track users who wait longer periods of time between visits. The tool relies on browser attributes such as resolution, user agent, plugins, canvas fingerprints, local storage, timezone, etc. Results indicate that FP-Stalker is able to track users for an average of $\approx$ 55 days, and over 100 days in the case of 26% of browsers.

2.2 DrawnApart

In March 2022, researchers improved upon these metrics by demonstrating their ability to fingerprint GPU devices with the DrawnApart attack[2]. Due to small manufacturing differences in same-model GPUs, different execution units on the GPU tend up varying slightly in terms of the time required for them to complete a task. By measuring these differences and combining the results, a GPU 'fingerprint' can be created. Since the execution unit to which a specific drawing task gets assigned is deterministic for the same workload, this fingerprint can be easily obtained using the WebGL API. When used in conjunction with FP-Stalker, DrawnApart increases the median tracking duration by 67%. With next generation GPU apis such as WebGPU (shipped in Chrome 113), DrawnApart's methodology can be made even more accurate.

2.3 Chromium GPU Stack

The Chrome GPU stack refers to the various layers involved in rendering graphics for the Chrome web browser using a computer's GPU. At the top of the stack is the browser compositor, which is responsible for compositing the graphical elements of a webpage into an image to be displayed on a user's screen. These graphical elements can be created using the graphics API, which is a programming interface allowing developers to access GPU hardware from the browser. Since GPUs vary widely between machines, Chrome will optionally select between multiple graphics APIs, including OpenGL, DirectX, and Vulkan depending on the system and hardware configurations of the user machine. This graphics API must be implemented by the system, where it will then have access to the GPU hardware device itself. For this project, we specifically focused on configurations which lead to the use of WebGL with Mesa on a weak GPU device.

2.3.1 WebGL

WebGL is a JavaScript API that adheres to the OpenGL ES 2.0 graphics standard, and enables web developers to utilize a computers GPU from within the browser. It is compatible with most major browsers, including Chrome, and has become a fundamental technology for the development of web-based applications such as games and visualizations.

2.3.2 Mesa

Mesa, also known as the Mesa 3D Graphics Library, is an open source implementation of OpenGL, Vulkan, and other graphics API specifications. It contains drivers for specific graphics card hardware configurations, which are connected to the top level API, which can be used by applications to access GPU functionality.

2.3.3 GPU Device

All modern laptop and desktop computers include some kind of GPU device, ranging from Intel integrated graphics chips to Nvidia RTX. Within a single GPU, there are multiple multiprocessor type segments each containing multiple execution units. These execution units are responsible for specific types of computations, and there is slight variance between the units themselves.

3 Implementation

3.1 Design

Our design is simple. DrawnApart works by abusing the fact that through WebGL, it is able to assign tasks directly to execution units. This means that it can time the execution time of certain tasks on specific execution units, and generate a data point from the average time for each. Since the execution units themselves will have minute differences both within a single GPU and across different instances of the same model GPU, these data points can be used to create a unique fingerprint.

To mitigate, we implement an extra step before the WebGL call is passed to the GPU driver: *randomization*. By randomizing the order in which tasks are assigned to execution units, we remove the ability of the site to associate execution units with times. This reduces the number of data points collectable such that the finest granularity at which timing data can be collected is for the GPU itself, a single data point. This data point will also have a much higher standard deviation than the per execution unit data points, since it will contain data from *all* execution units, which we have already established to have measurable differences in execution time. For these reasons, any fingerprint created on top of our randomization scheme will be much more difficult to uniquely link with a user, since fingerprints created for the same model GPU will be much more similar across instances.

3.2 DrawArrays / DrawElements

In WebGL 1.0, there are two commands that execute code on the GPU: DrawArrays and DrawElements. DrawArrays runs a shader function on an array of vertexes representing points, lines, or triangles. DrawElements is similar, but it allows for selecting specific vertexes from the array (and their order) using a separate index array. Consequently, any call to DrawArrays can be replaced with an equivalent call to DrawElements. The following two segments of WebGL pseudocode, for example, are functionally identical.

```
BufferData(VERTEX_BUFFER,
           [v1, v2, v3, v4, v5, v6]);
DrawArrays(...);

BufferData(VERTEX_BUFFER,
           [v1, v2, v3, v4, v5, v6]);
BufferData(INDEX_BUFFER,
           [0, 1, 2, 3, 4, 5]);
DrawElements(...);
```

Depending on the draw mode (specified on a call to DrawArrays or DrawElements), the vertexes (or indexes) can represent either points, lines, or triangles. Although some draw modes reuse vertexes across multiple lines or triangles, the main draw modes create lines from ever two vertexes and triangles from every three vertexes, respectively; in the code segments above, the draw mode LINES would create three lines L1(v1, v2), L2(v3, v4), and L3(v5, v6), and the draw mode TRIANGLES would create two triangles T1(v1, v2, v3) and T2(v4, v5, v6). Crucially, for these draw modes, the indexes can be shuffled (with a stride of 1 for points, 2 for lines, and 3 for triangles) with no effect on the output. Additionally, the order in which a vertex gets assigned to an execution unit on the GPU depends on its position in the index buffer, which is not visible to the shader code. We use these facts as the basis for our implementation.

We made modifications to several functions that forward WebGL calls from Chrome's renderer process to the GPU driver. Simplified pseudocode for the modified functions is shown below.

3.2.1 DrawArrays

We simply create an index array using *start* and *count*, then pass the array to DoShuffledDraw (described later), which sends a DrawElements call to the GPU driver.

```
DrawArrays(GLenum mode, int start, int count) {
    vector<int>* indexes =
            CreateAscending(start, count);
    DoShuffledDraw(mode, indexes, count);
    free(indexes);
}
```

3.2.2 BufferDataImpl

BufferDataImpl is a helper function that is called to write data to a buffer. Because the CPU cannot efficiently read the contents of a WebGL buffer, we must copy the data being written and keep track of it using a global mapping of buffer objects to index arrays.

```
BufferDataImpl(GLenum buffer_type,
               vector<int> indexes,
               int count) {
    \\ existing code
    if (buffer_type == INDEX_BUFFER) {
        WebGLBuffer* buffer =
                GetCurrentBuffer(INDEX_BUFFER);
        index_buffer_map_.Set(
                buffer, indexes.Copy());
    }
}
```

3.2.3 DrawElements

We retrieve the index array from the global mapping, then pass it to DoShuffledDraw.

```
DrawElements(GLenum mode, int count, int offset) {
    WebGLBuffer* buffer =
            GetCurrentBuffer(INDEX_BUFFER);
    vector<int>* indexes =
            index_buffer_map_.Get(buffer);
    indexes = indexes.CopyRange(
            offset, offset + count);
    DoShuffledDraw(mode, indexes, count);
    free(indexes);
}
```

3.3 DoShuffledDraw

The DoShuffledDraw function is a wrapper function for a randomized draw operation. Given some indexes to randomize, it performs pre-processing operations and then binds the shuffled indexes to the global state, before calling DrawElements.

```
DoShuffledDraw(GLenum mode,
               vector<int>* indexes,
               int count) {
    GLenum unrolled_mode;
    int unrolled_count = UnrollIndexesIfNecessary(
            mode, indexes, &unrolled_mode);
    ShuffleIndexes(unrolled_mode,
                   unrolled_count,
                   indexes);
    BindBuffer(INDEX_BUFFER,
               global_index_buffer_);
    BufferData(INDEX_BUFFER,
               indexes,
               unrolled_count);
    Context->DrawElements(
```

```
            unrolled_mode, unrolled_count, 0);
    RestoreState();
}
```

DoShuffledDraw calls two helper functions that we created for preprocessing and shuffling the index arrays, which are described below.

3.3.1 UnrollIndexesIfNecessary

WebGL implements 6 different GLenum types for drawing:

- Points
- Lines
- Triangles
- Line Strips*: These are sets of points $P_1, P_2, ...P_n$ such that for all adjacent points there is a line connecting the them.
- Line Loops*: These are similar to line strips, with the additional requirement that the first and last points are connected.
- Triangle Fans*: These are a set of connected triangles, all sharing one central vertex.
- Triangle Strips*: These are sets of triangles represented with shared vertices.

The types marked with * are *derived* from the base types, and therefore have alternative representations composed of only the base types, which are slightly less memory efficient. The issue is that when using the derived type representations, because they are optimized to save memory by sharing points, it is not possible to shuffle the indices without changing the implied connections. To counteract this, we must first 'unroll' them into their base type representation so that we eliminate this vertex sharing, and allow our shuffle to work properly.

3.3.2 ShuffleIndexes

ShuffleIndexes uses a helper function defined by the interface shown below.

```
template <int stride>
ShuffleIndexesHelper(int* indexes, int count);
```

Our shuffle function shuffles variable length elements in-place using the Fisher-Yates shuffle algorithm. This implementation introduces overhead in that individual elements must be moved using *memcpy*. However, since the number of bytes copied is defined by the *stride* template field, the compiler optimizes away this overhead.

The ShuffleIndexes wrapper function used by DrawArrays and DrawElements sets the stride of the shuffle based on the draw mode.

```
ShuffleIndexesHelper(int* indexes,
                     int count,
                     GLenum draw_mode) {
    switch (mode) {
        case POINTS:
            ShuffleIndexesHelper<1>(
                    indexes, count);
            break;
        case LINES:
            ShuffleIndexesHelper<2>(
                    indexes, count);
            break;
        case TRIANGLES:
            ShuffleIndexesHelper<3>(
                    indexes, count);
            break;
    }
}
```

4 Results

4.1 Experimental Setup

For this project, we had limited access to hardware on which to test our designs. We used 2 machines with the following specifications:

- Apple M1 Pro with 16 core GPU and 32GB RAM
- 12th gen Intel i5 with Intel UHD 770 Graphics and 16GB RAM

4.2 Baseline

Prior to implementing our design, we collected results from the DrawnApart attack demo running on the two test machines. We focused on two simple data metrics, in order to understand the unique fingerprints of each GPU. Since we used two entirely different devices for comparison, the difference in fingerprint is significantly more drastic than simple manufacturing differences, and as a result we are able to visualize the difference in fingerprints.

In figure 1 we show the GPU fingerprints obtained by repeatedly running the DrawnApart demo attack. Results are as expected, showing a clear difference between the fingerprint of the M1 GPU and the Intel graphics chip. The M1 GPU also seems to show much greater variation between execution units.

In figure 2 we show the standard deviation of execution times measured for a single execution unit. This value is significantly higher on the M1 GPU. Higher standard deviations make the DrawnApart attack less accurate as it is more difficult to measure minute differences in execution unit performance when the range of execution times is large.

4.3 Randomized EU Assignment

After implementing our design, we re-ran the DrawnApart Benchmarks to see if we had made our desired improvements. Our results are included in the below figures. When running on the M1 device, we saw a much smoother curve shape. This indicates positive results, as the imperfections between

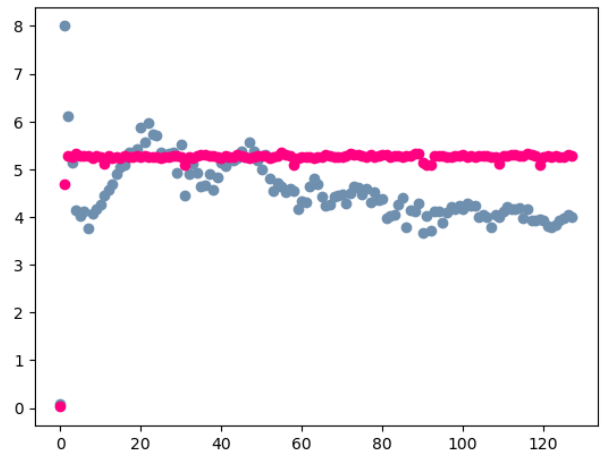

Figure 1. Average execution time per EU for the DrawnApart workload run on two different GPUs

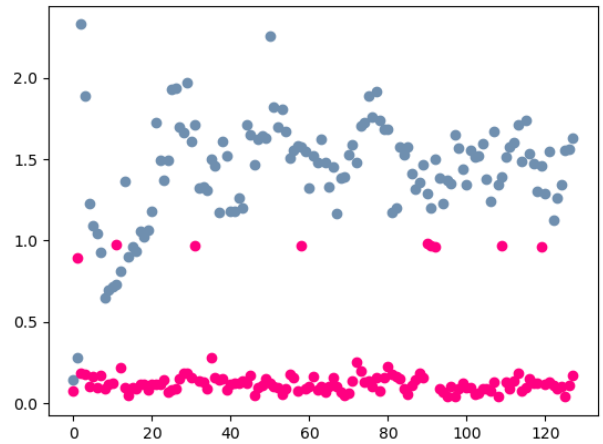

Figure 2. Standard deviation of execution time per EU for the DrawnApart workload run on two different GPUs

EUs are no longer visible, and instead we only see the characteristic M1 GPU curve expected of any same-model device (see fig. 3). This fingerprint is likely much more difficult to distinguish from other M1 GPUs. We still see relatively high standard deviation between logical buffer indexes (DrawnApart's expected EUs) in fig. 4, but this is expected due to the unique 'speedup' pattern of the M1 GPU due to it's advanced design, and not a reflection of our algorithm.

We see the best results on the i5 machine. Before implementing our design, all EUs took approximately the same time to execute with slight deviations, indicating the expected imperfections between units. After the implementation of the shuffle function, these imperfections disappear almost entirely in fig. 5, and it is not possible to distinguish between EUs only given the DrawnApart results. We also see that the standard deviation between logical buffer indices is

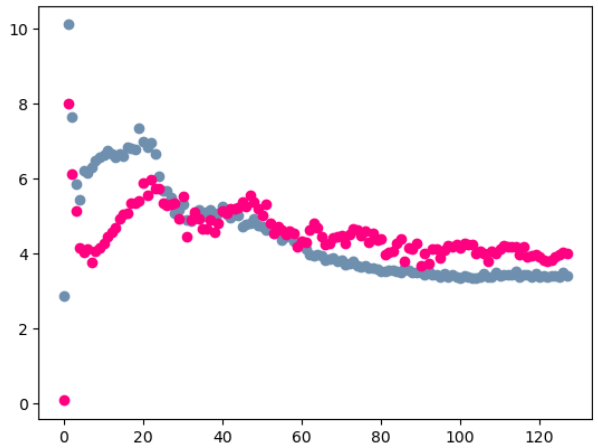

Figure 3. Average run-time per logical buffer index on the M1 device before (pink) and after (blue) implementation of Pumice

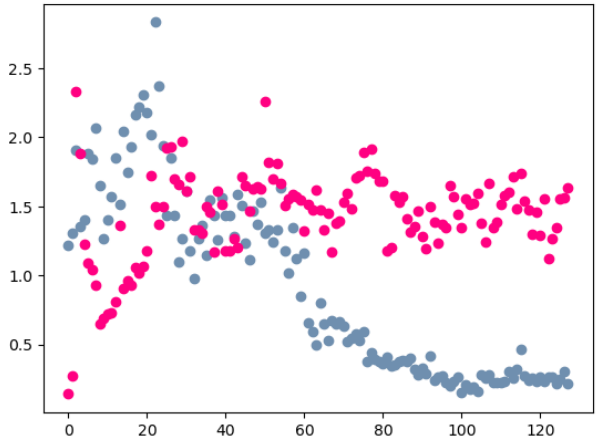

Figure 4. Standard deviation per logical buffer index on the M1 device before (pink) and after (blue) implementation of Pumice

much more constant, and on average larger, which makes fingerprinting the GPU accurately more difficult.

4.4 Performance Impact

Due to the implementation of our design, we add additional memory accesses and GPU memory migrations as overhead every time we submit a *drawElements* command to the GPU. This is due to the *memcpy* operations associated with the shuffle function and the need to re-migrate index buffers to the GPU for every draw. There is a noticable performance decrease when using a version of the chromium browser built with our solution. While we argue that this is forgivable since browser workloads are not typically GPU intensive, we admit this is a place for improvement.

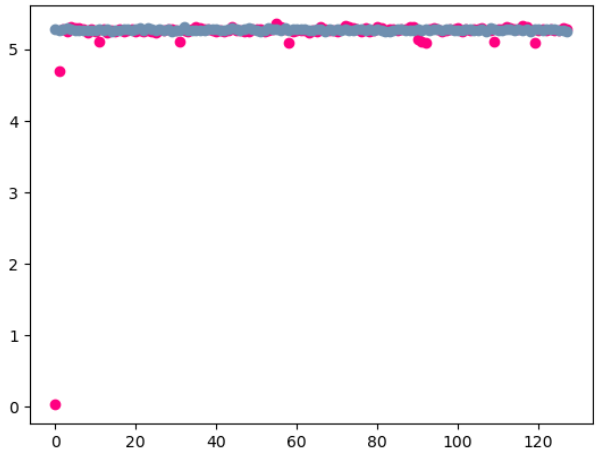

Figure 5. Average run-time per logical buffer index on the i5 device before (pink) and after (blue) implementation of Pumice

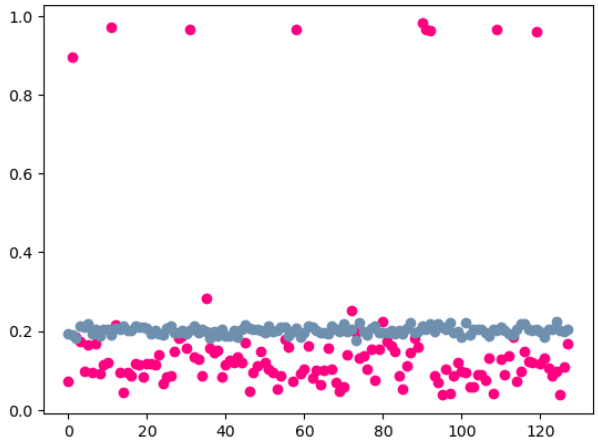

Figure 6. Standard deviation per logical buffer index on the i5 device before (pink) and after (blue) implementation of Pumice

4.4.1 Caching Shuffles

Since the overhead is found in the copy and shuffle portion of the code, caching shuffle results on the GPU for a single session could provide a very large reduction in overhead. Our current proof of concept design requires 2 copies between the GPU and CPU due to the registering and unregistering of the shuffled buffer. In order to preserve state, we are currently restoring the index buffer as it was expected by the user, however, we have seen no evidence that this is necessary given that users should not be reading from the registered buffer. Removing the unnecessary registering, and only re-registering if our buffer has been overwritten seems to be a promising way to improve performance. However, we were unable to successfully implement this speedup given the timeline of the project, so we leave it to future work for verification of this feature.

5 Conclusion

Protecting users right to privacy on the internet is crucial given the pervasive attempts at collection and storage of personal data by companies and organizations. Mitigating GPU fingerprinting is just one such way in which we can contribute to this effort, but the strong financial incentive to violate user privacy means that new fingerprinting techniques will likely continue to emerge. Frameworks such as WebGL and WebGPU are impressive in their ability to provide web developers access to hardware in a way that has not been possible in the past, but such allowances open the door to attacks that must be carefully sought out and monitored. We hope to continue contributing to ongoing efforts to prioritize user privacy and improve the online experience by providing a means to address a specific type of browser fingerprinting, and thus enhancing privacy for users.

6 Metadata

The presentation of the project can be found at: https://drive.google.com/file/d/1gTJGgOKHVGGd7C_P2DxM0trjB1Im5v4r/view?usp=sharing
The code/data of the project can be found at: https://github.com/rohitc33/chromium

References

[1] Krishnar, G. Nothing private. https://github.com/gautamkrishnar/nothing-private, 2016.
[2] Laor, T., Mehanna, N., Durey, A., Dyadyuk, V., Laperdrix, P., Maurice, C., Oren, Y., Rouvoy, R., Rudametkin, W., and Yarom, Y. Drawnapart: a device identification technique based on remote gpu fingerprinting. *arXiv preprint arXiv:2201.09956* (2022).
[3] Molla, R. The cost of an ad-free internet: $35 more per month, 2019.
[4] Pawtocol. How much is user data worth?, 2022.
[5] Vastel, A., Laperdrix, P., Rudametkin, W., and Rouvoy, R. Fp-stalker: Tracking browser fingerprint evolutions. In *2018 IEEE Symposium on Security and Privacy (SP)* (2018), IEEE, pp. 728–741.